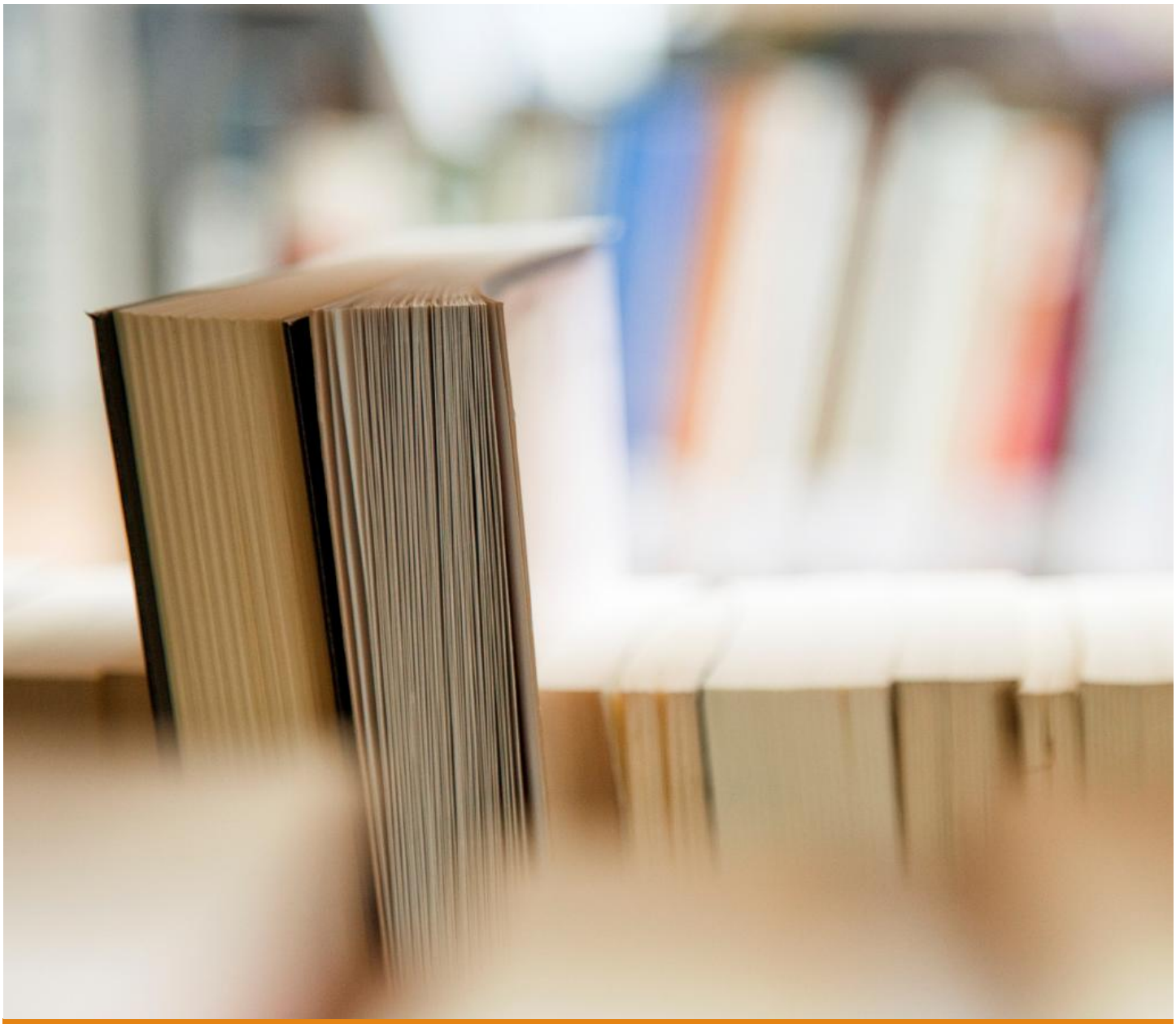


# SQL REPORT

## CONTROLLED LAB ENVIROMENT

KARABO MANUEL KAMOGELO MOLAPO

5/19/26

Title: Examining the database

Risk rating: High (CVSS 8,5)

## Summary:

A UNION-based SQL Injection attack was used to identify the database table containing user credentials and extract usernames and passwords.
The retrieved administrator credentials were then used to successfully authenticate into the application as the administrator user.

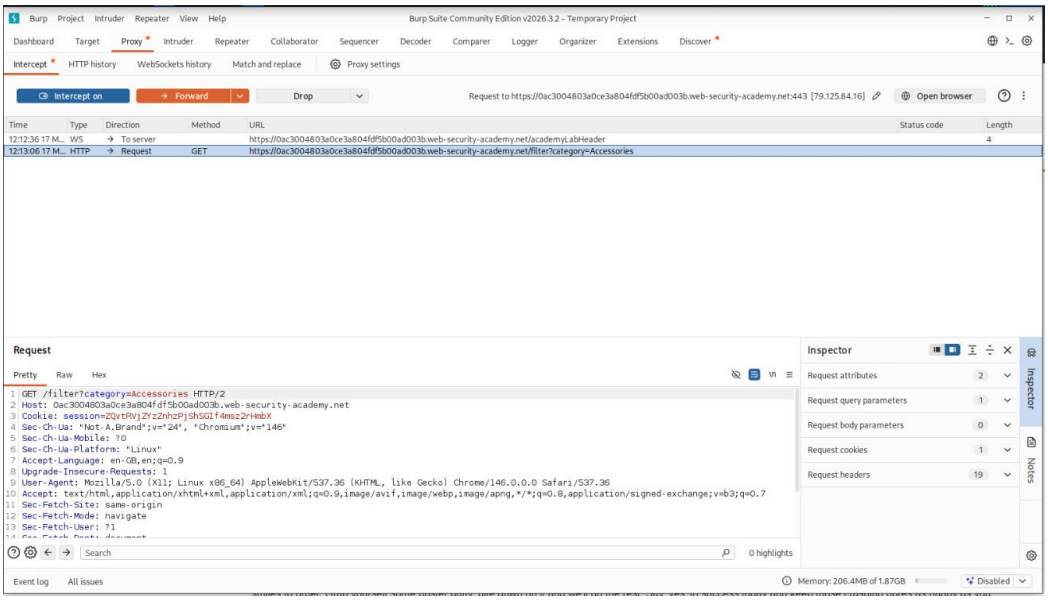
## Background:

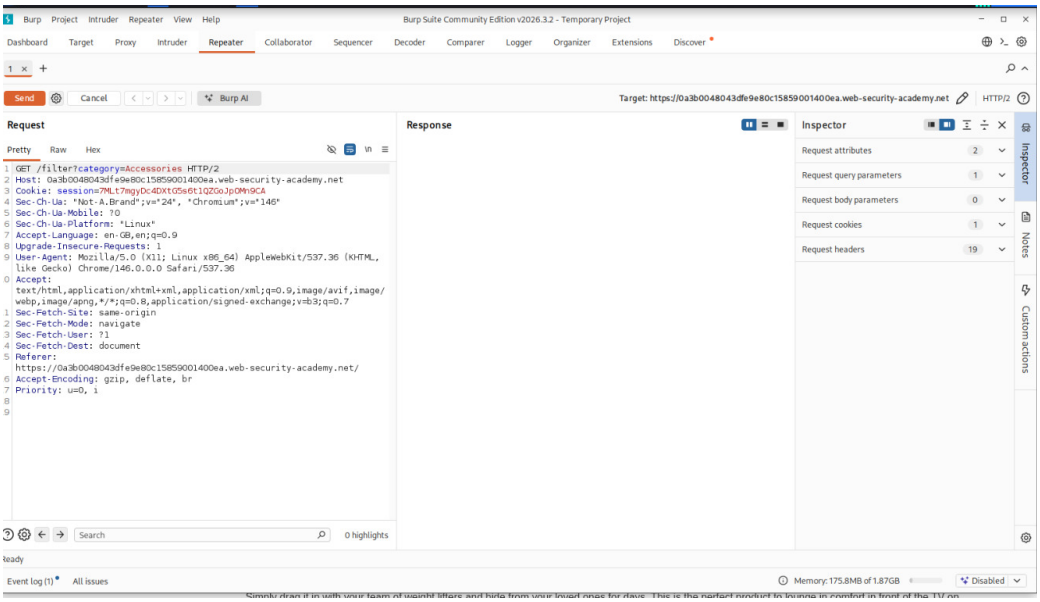
The application contains a SQL Injection vulnerability within the product category filter, allowing user input to be directly processed by the backend database query.
This vulnerability enables attackers to manipulate SQL statements and retrieve unintended data from other database tables.

## Technical details and Evidence:

Burp Suite Community Edition was used to intercept and modify HTTP traffic between the browser and the application. The vulnerable GET request containing the category parameter was captured and analyzed during testing.

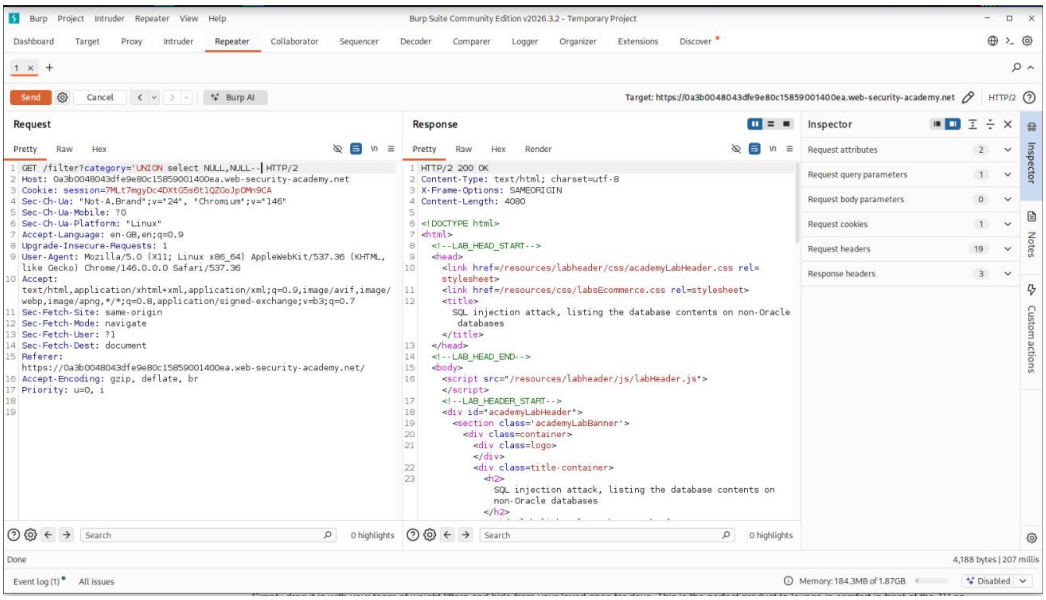
The intercepted GET request was then sent to the Repeater tab in Burp Suite, allowing the request to be manually modified and resent to the server.



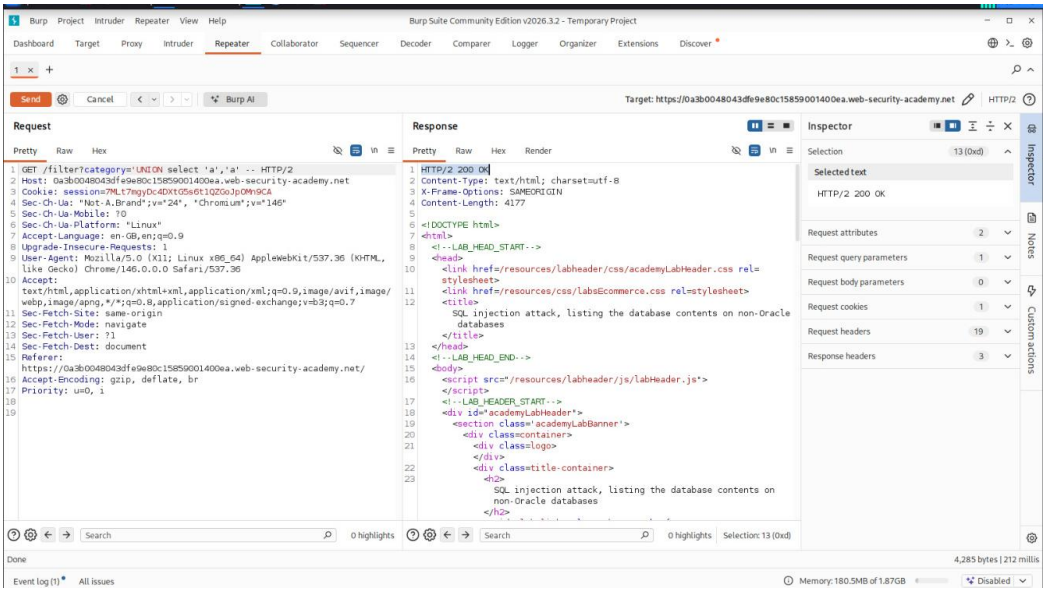

A SQL injection payload was inserted into the vulnerable category filter parameter, as the application indicated that this input was susceptible to SQL Injection attacks. The payload ' UNION SELECT NULL, NULL-- was used to determine the number of columns returned by the original SQL query.

The application responded successfully to the payload, confirming that the query structure contained two columns. Matching the correct number of columns was necessary to perform further UNION-based SQL Injection attacks and retrieve data from other database tables.




The payload ' UNION SELECT 'a', 'a'-- was inserted into the vulnerable category parameter to identify the data types of the returned columns.

The application processed the payload successfully, confirming that both columns accepted string-based input. This indicated that the columns were compatible with text data types, allowing further retrieval of textual information from the database.



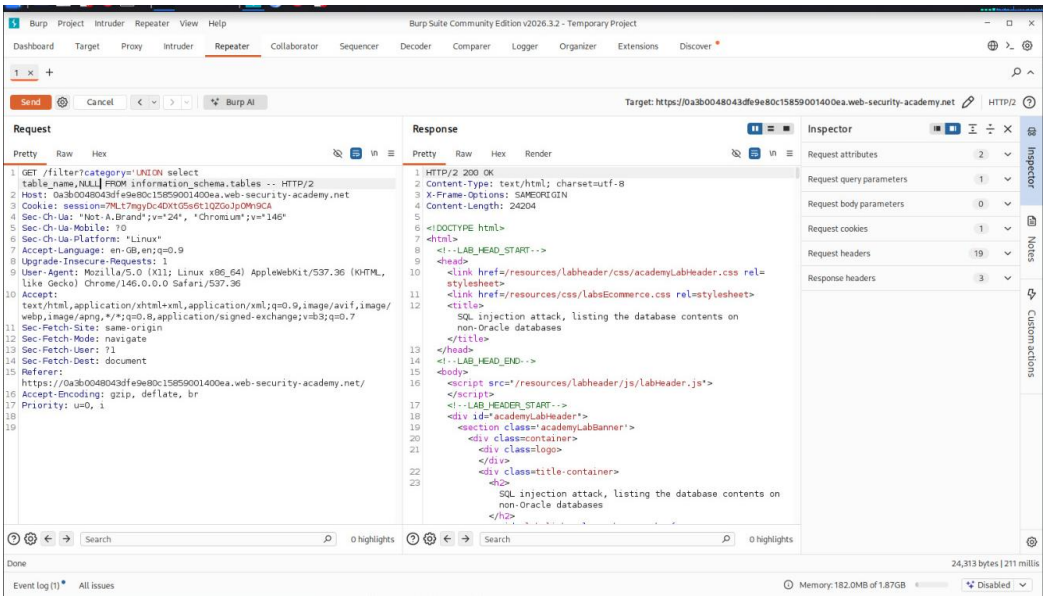

Another payload, 'UNION SELECT version (), NULL--, was inserted into the vulnerable category parameter to retrieve information about the backend database.

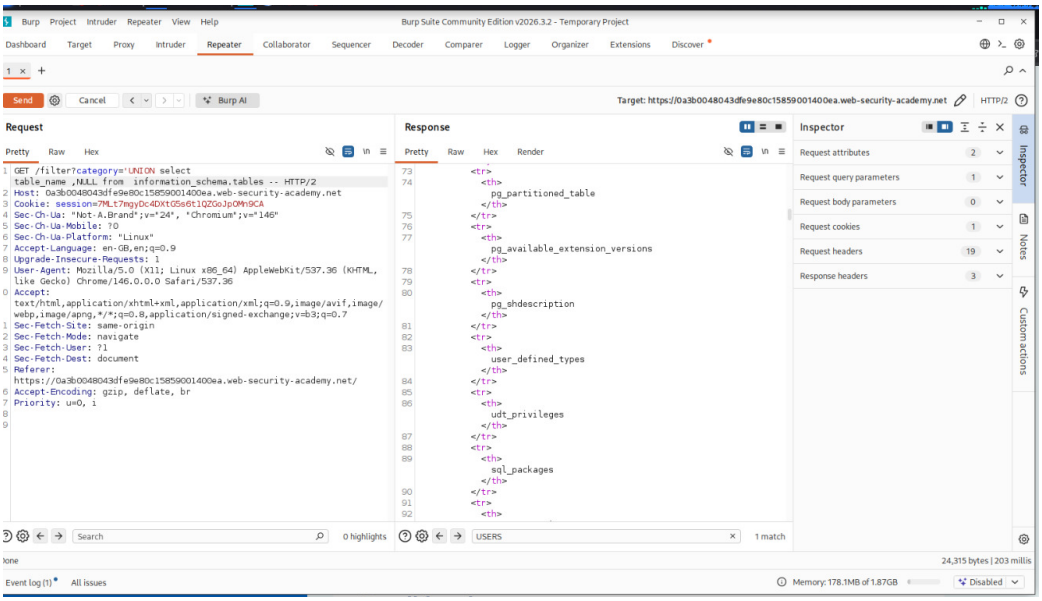
The application returned the database version successfully, confirming that the web application was using a PostgreSQL database management system.

Knowing that it uses the PostgreSQL database.



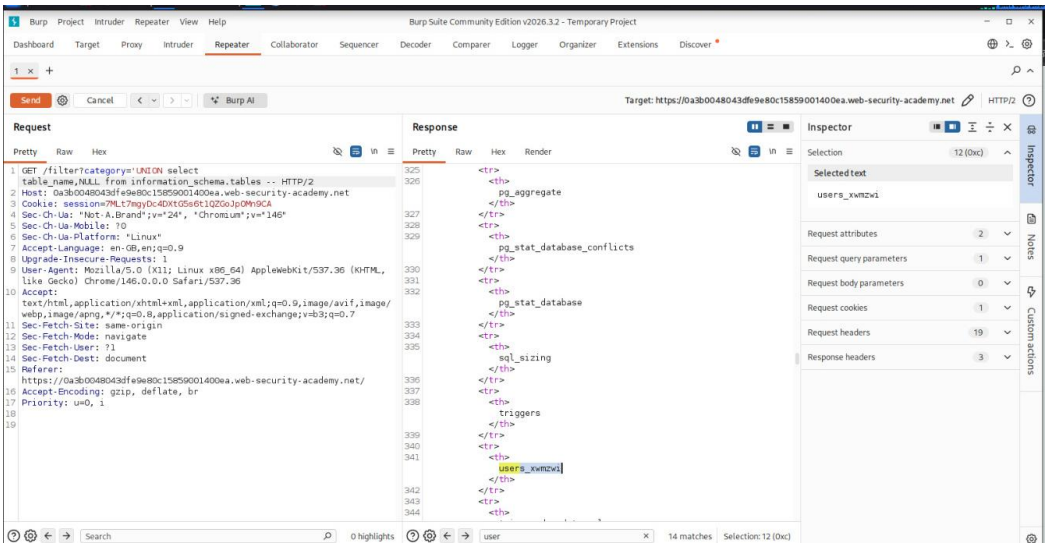

I used the payload on PostgreSQL to determine the table name.





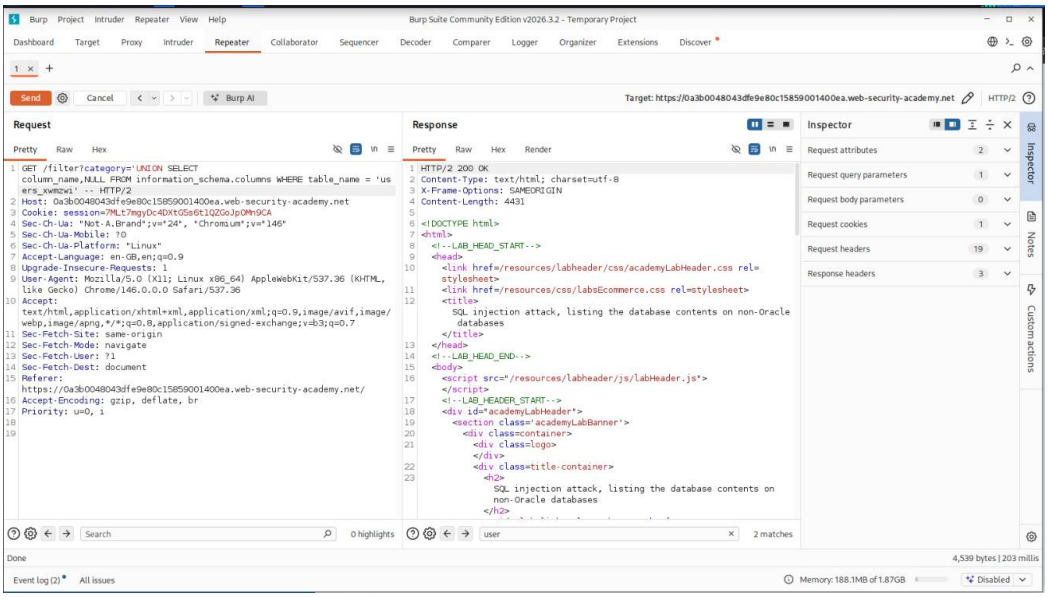

Further SQL injection enumeration was performed to identify database tables within the PostgreSQL database. The query results displayed multiple table names returned from the database schema.

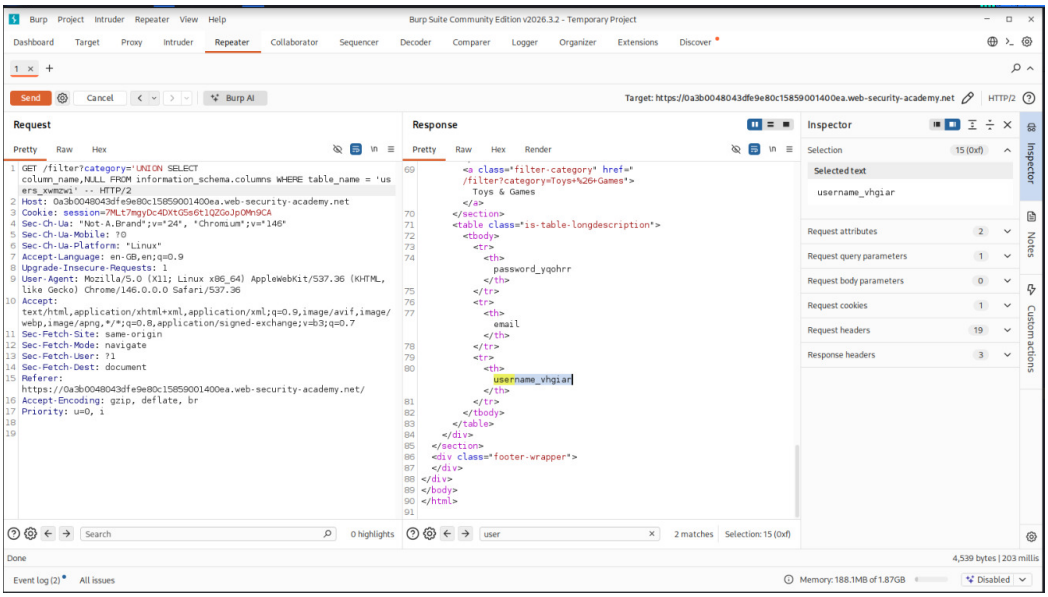
Among the discovered tables was the users table, which was identified as likely containing authentication-related information such as usernames and passwords.




After identifying the users' table, further enumeration was conducted to discover the column names contained within the table.

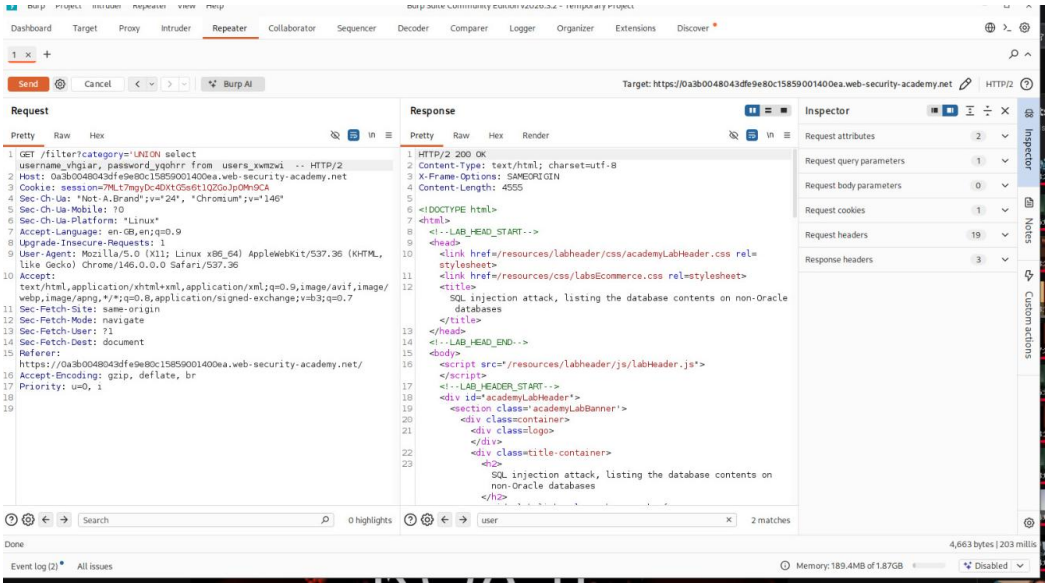
Identifying the column names was necessary to determine which fields stored sensitive information such as usernames and passwords, enabling successful data extraction through UNION-based SQL Injection.

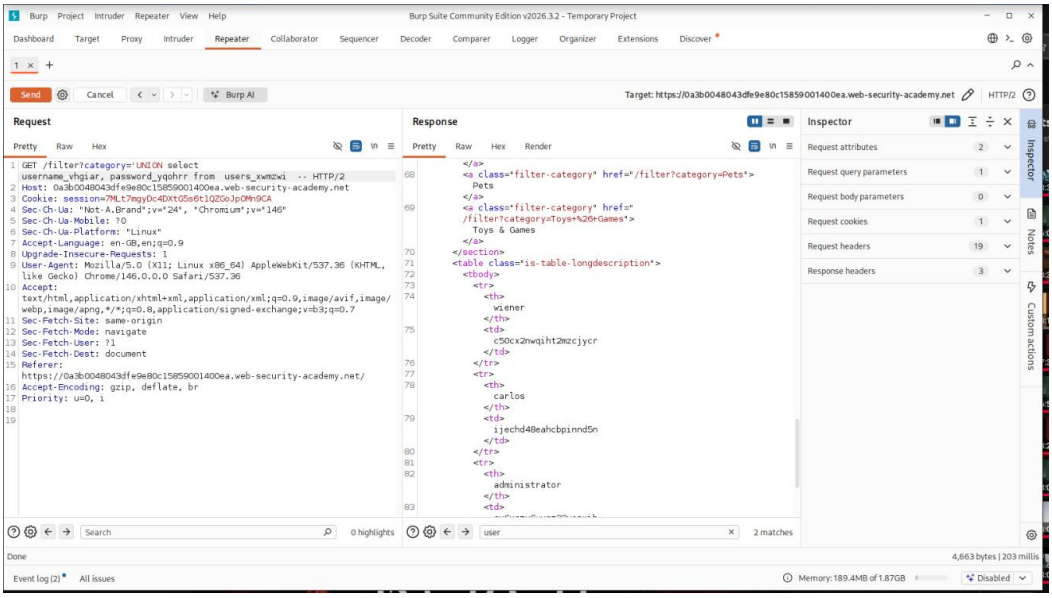



After identifying the relevant column names, a UNION-based SQL injection payload was used to retrieve user's credential data from the database table. The payload ' UNION SELECT username, password FROM users-- was inserted into the vulnerable parameter.

The application returned the contents of the username and password columns from the users table, successfully exposing user credentials stored within the database.





## Impact

The SQL Injection vulnerability allowed unauthorized access to sensitive information stored within the database, including usernames and passwords from the users table. An attacker could use the extracted administrator's credentials to gain privileged access to the application and potentially compromise additional user accounts and data.

Successful exploitation may also enable attackers to enumerate database structures, retrieve confidential information, bypass authentication mechanisms, and perform further attacks against the application or backend database.

## Remediation

Implement parameterized queries (prepared statements) to ensure user input is treated strictly as data and not executable SQL commands. Avoid dynamically constructing SQL queries using unsensitized input from users.

Additionally, enforce strong input validation, apply least-privilege access controls on database accounts, and use secure error handling to prevent database information disclosure. Regular security testing and code reviews should also be performed to identify and remediate SQL Injection vulnerabilities before deployment.

## Reference

https://portswigger.net/web-security/sql-injection/cheat-sheet.
https://0aed00800395ae0081f83569005900e3.web-security-academy.net/